Listing 2.6 Using prepared statements

```
public JSONObject createSpace(Request request, Response
response)
throws SQLException {
var json = new JSONObject(request.body());
var spaceName = json.getString("name");
var owner = json.getString("owner");
return database.withTransaction(tx -> {
var spaceId = database.findUniqueLong(
"SELECT NEXT VALUE FOR space_id_seq;");
database.updateUnique(
"INSERT INTO spaces(space_id, name, owner) " + #A
"VALUES(?, ?, ?);", spaceId, spaceName, owner); #A
response.status(201);
response.header("Location", "/spaces/" + spaceId);
return new JSONObject()
.put("name", spaceName)
.put("uri", "/spaces/" + spaceId);
});
```

#A Use placeholders in the SQL statement and pass the values as additional arguments.

Now when your statement is executed, the database will be sent the user input separately from the query, making it impossible for user input to influence the commands that get executed. Let’s see what happens when you run your malicious API call. This time the space gets created correctly – albeit with a funny name!

```
$ curl -i -d "{\"name\": \"', ''); DROP TABLE spaces; --\",
[CA]\"owner\": \"\"}" http://localhost:4567/spaces
HTTP/1.1 201 Created
```